Field Hours

CSCE 642 Deep Reinforcement Learning

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Markov decision processes

You write an MDP as states, actions, a transition kernel, and a reward. The Markov property says the next state depends on the past only through the current state and the action you just took. With discount 0.9, a reward of 1 that arrives 10 steps from now is worth 0.9 to the tenth power, about 0.35. Distant payoffs shrink before you pick an algorithm.

You also choose a horizon. A finite horizon of 5 makes the policy a function of state and time, because the last step has no future left. An infinite discounted horizon lets one stationary policy cover every time. If every reward sits between 0 and 1 and the discount is 0.9, the return is at most 1 divided by 1 minus 0.9, which equals 10. That bound keeps a value table finite when the episode never ends.

What you store in an MDP.

States and actions

Transition kernel

Reward each step

Discount on later reward

A two-state return

1. From A, the only action moves to B with reward 0. In B, the only action stays in B with reward 4. The discount is 0.5.

2. The return from B is 4 plus 0.5 times 4 plus 0.25 times 4 and so on, which sums to 4 divided by 0.5, so 8.

3. From A the return is 0 plus 0.5 times 8, so 4. You do not add the reward at B a second time outside that discount.

Problem 1. Every reward is at most 2 and the discount is 0.5. What is the largest possible return?

Answer. 4. The infinite sum of 2 times 0.5 to the k is 2 divided by 1 minus 0.5.

Problem 2. You switch to a horizon of 1, starting in A, under the same transitions. What return do you collect, and why does time matter?

Answer. 0. The only reward inside the horizon is the immediate 0 for leaving A. The value of B is never used, because you do not act there. A horizon of 1 makes value depend on remaining time.

Lecture 2 · 90 minutes

Value iteration

The Bellman optimality backup sets a state's value to the best action's expected reward plus the discount times the expected value of the next state. You turn that equation into value iteration by applying the backup to every state from a table of zeros. On a grid with 4 states and discount 0.9, one sweep rewrites all 4 entries. The largest change tends to shrink by about a factor of 0.9 each sweep, because the backup is a contraction.

You stop when the biggest absolute change in any state falls below a tolerance you set, often 0.01 when rewards are near 1. The greedy policy picks, in each state, the action with the best one-step backup, and it is optimal once the values are exact. If you stop early, leftover error can still flip the best action wherever two actions differ by less than that leftover. You compare the action gap with the residual, not the residual alone.

One value-iteration sweep.

1. Read the value table

2. Backup every state

3. Keep the best action

4. Stop on a small change

Two states, old table held fixed

1. State H pays reward 2 and stays in H. State L pays 0 and moves to H. Discount 0.5. Start both values at 0, and use the previous table for the whole sweep.

2. After one sweep, V(H) is 2 plus 0.5 times 0, so 2, and V(L) is 0. After the next sweep, V(H) is 2 plus 0.5 times 2, so 3, and V(L) is 0.5 times 2, so 1.

3. The fixed point solves V(H) = 2 plus 0.5 V(H), so V(H) is 4, and V(L) is 0.5 times 4, so 2. One sweep does not land there.

Problem 1. At the fixed point of this two-state backup, what is V(L)?

Answer. 2. V(H) is 4, and L moves to H with reward 0, so its value is 0.5 times 4.

Problem 2. Your tolerance is 0.5. In one state the two action backups are 3.2 and 3.4. Can you trust the greedy choice?

Answer. No. The gap is 0.2, which is smaller than the 0.5 leftover you still allow. A later sweep can reverse which action looks better.

Lecture 3 · 90 minutes

Policy iteration

Policy evaluation freezes a policy and solves the linear Bellman expectation equation, the backup without the max. With 3 states you get 3 equations and 3 unknown values. You may iterate that backup or solve the linear system once. Either way you obtain the value of the policy you froze, not the optimal value. A policy you have not evaluated does not yet have a return you can quote.

Policy iteration then improves the policy. In every state you replace the action by the one whose one-step backup is largest under the values you just computed. On a table with 2 actions per state, one improvement pass compares 2 backups in each state. If no action changes, you have an optimum and you stop. You then evaluate the new policy before you improve again, because a backup from old values is not the new policy's return.

Policy iteration until the actions hold.

Freeze the policy -> Solve the linear values -> Greedy improvement -> Stop if the policy holds

Stay or leave

1. In S, stay pays 1 and remains in S. Leave pays 0 and goes to T. In T the only action pays 3 and stays. Discount 0.5. Start by evaluating the policy that stays in S.

2. V(S) = 1 plus 0.5 V(S), so V(S) is 2. V(T) = 3 plus 0.5 V(T), so V(T) is 6. The stay backup is 2. The leave backup is 0.5 times 6, so 3. Improvement switches S to leave.

3. Under leave, V(S) is 0.5 times 6, so 3. Stay's backup against that value is 1 plus 0.5 times 3, so 2.5, which is worse. The improved policy is stable.

Problem 1. After the first evaluation, which action does improvement pick in S, and what is its backup?

Answer. Leave, with backup 3. Stay's backup is 2.

Problem 2. You keep the stay policy and never improve. By how much is V(S) worse than under the improved policy?

Answer. By 1. Stay evaluates to 2. Leave evaluates to 3.

Lecture 4 · 90 minutes

Q-learning without a model

You do not have the transition table. Temporal-difference learning updates a value from one transition: the target is the reward you just saw plus the discount times the value of the next state, and you step a fraction alpha of the way toward that target. With alpha 0.5, an old value of 0 and a target of 4 becomes 2 after one update. The same transition a second time moves the value from 2 to 3.

Q-learning stores one number per state-action pair and targets the best next action, not the action you take next. That off-policy backup lets a behavior policy with epsilon 0.1 gather data while the stored values still chase the greedy action. With 4 actions, epsilon 0.1 picks a uniform random action about 1 time in 10. If epsilon is 0 and every entry starts at 0, a fixed tie break never tries the other 3 actions.

Two one-sample backups.

TD on a state

Target uses next value

Samples the current policy

One learning rate

Q-learning

Target uses max next Q

Behavior can differ

Epsilon forces tries

Two identical Q updates

1. Q(s, a) starts at 0. You take a, receive reward 2, and land where the best next Q is 4. Discount 0.5. Alpha 0.5.

2. The target is 2 plus 0.5 times 4, so 4. The new Q is 0 plus 0.5 times (4 minus 0), so 2.

3. The same transition again keeps the target at 4. The new Q is 2 plus 0.5 times (4 minus 2), so 3. You never stored the transition itself.

Problem 1. What is Q(s, a) after the two updates in the example?

Answer. 3.

Problem 2. Epsilon is 0, every Q starts at 0, and ties break toward action 1. How many of the other actions do you sample?

Answer. None. You always take action 1, so the other actions stay at 0. The usual convergence story needs every pair tried enough times, and that does not happen.

Lecture 5 · 90 minutes

Function approximation

A table cannot store a state that is an 84 by 84 image. You replace the table by a parameterized function, here a network with about 2 million weights, and you train those weights so the output matches a Bellman target. The update is no longer a write into one cell. A step that cuts error on the current transition can raise error on another transition that shares the same weights.

Three ingredients together are the usual divergence recipe: bootstrapping from your own next-state estimate, off-policy data, and a nonlinear function class. Dropping one of them changes the failure you should expect. A linear function, on-policy temporal difference, and 16 features can still be studied as a contraction in a weighted norm. A deep Q network keeps all 3 ingredients, so you add a replay buffer and a second target network instead of hoping the raw update stays put.

The table becomes a shared function.

Image or vector state

Shared parameterized function

Bellman target

One step touches many states

One weight for two states

1. Both states read the same weight w, and you want targets 0 and 2. The constant that minimizes

average squared error is the mean, (0 plus 2) divided by 2, so w = 1. Squared error is 1 at each state.

2. A step that only sees the second state moves w toward 2. From 1, adding 0.5 times the error (2 minus 1) sets w to 1.5.

3. Error on the first state grows from 1 to 1.5, while error on the second falls from 1 to 0.5. A table would have edited only the cell you wrote.

Problem 1. What single weight minimizes average squared error on the targets 0 and 2?

Answer. 1, the mean of the two targets.

Problem 2. Why can a step that is right for the second state increase error on the first?

Answer. Both states use the same weight. Moving w from 1 to 1.5 matches the target 2 better and the target 0 worse. Sharing is the mechanism, not a bug in the arithmetic.

Lecture 6 · 90 minutes

Deep Q networks

A deep Q network outputs one number per action. You store each transition in a replay buffer and draw a minibatch of 32 older transitions, so the update is not tied to the latest frame alone. That weakens the strong time correlation of consecutive frames. The target uses a copy of the weights that you refresh every 1000 steps, so the regression label does not chase the same weights you are moving on this step.

You score one transition by the squared gap between online Q and a frozen target. The target is the reward plus the discount times the best next frozen Q, and you do not differentiate through that max. With rewards clipped into [-1, 1] and discount 0.99, a target has absolute value at most 1 divided by 0.01, which is 100. Clipping, not the raw game score, sets that scale for the fit.

One deep Q update.

Store the transition -> Sample a batch of 32 -> Build frozen targets -> Step the online weights

A target that does not move

1. Online Q(s, a) is 1.5. The frozen max next Q is 2.0. The reward is 0. The discount is 0.5.

2. The target is 0 plus 0.5 times 2.0, so 1.0. The squared error is (1.5 minus 1.0) squared, so 0.25.

3. You step the online weights to reduce that 0.25. The frozen copy stays at 2.0 until the next refresh, which in this design is every 1000 steps, not every step.

Problem 1. Online Q is 1.5, reward is 0, discount is 0.5, and the frozen max is 2. What are the target and the squared error?

Answer. The target is 1.0. The squared error is 0.25.

Problem 2. You copy the target network every step instead of every 1000 steps. What did the regression lose?

Answer. A label that stays fixed while you fit it. The target now moves with the online weights, so you chase a changing number instead of regressing onto a snapshot.

Lecture 7 · 90 minutes

Policy gradients

A stochastic policy assigns probabilities to actions. The policy gradient theorem writes the gradient of expected return as an expectation of the return times the gradient of the log probability of the action you sampled. REINFORCE multiplies that log-probability gradient by the full episode return. With a return of 10 and a learning rate of 0.01, the update scale is 0.1 times that gradient, and a large return on a poor action still increases its probability.

An actor-critic replaces the full return by a one-step temporal-difference error, the advantage, so you do not wait for the episode to end. The critic is a value network and the actor is the policy network. A state-only baseline does not change the expected gradient, and it cuts variance. PPO clips the probability ratio at 0.2 on either side of 1, so one batch cannot multiply an action probability by more than a factor of 1.2.

An actor-critic step.

1. Sample an action

2. Form the advantage

3. Scale the log probability

4. Clip the ratio in PPO

Return versus a baseline

1. Two actions have probabilities 0.8 and 0.2. You sampled the rare action. The episode return is 5, with the discount already folded in.

2. Plain REINFORCE scales the log-probability gradient of that action by 5, so a positive return raises its probability.

3. A state baseline of 4 scales the same gradient by 5 minus 4, so 1. The direction is unchanged and the step is smaller. That subtraction is the actor-critic advantage in the one-step case when the critic matches the baseline.

Problem 1. The return is 5 and the state baseline is 4. What number scales the log-probability gradient?

Answer. 1, the advantage 5 minus 4.

Problem 2. The PPO clip is 0.2 and a batch wants a probability ratio of 1.8 on an action with positive advantage. What ratio does the clipped objective use?

Answer. 1.2. The allowed window is 0.8 to 1.2, and 1.8 lies above it. With positive advantage the objective uses the upper end, 1 plus 0.2.

Lecture 8 · 90 minutes

What you can train and check

You can now write a small MDP, compute a discounted return, and run value iteration or policy iteration when

the table fits in memory. You can also explain a Q-learning update with a learning rate of 0.5, and you can say why a deep Q network adds a replay buffer plus a target network copied every 1000 steps. Given a policy-gradient step, you can point at the return or the advantage that scales the log probability.

You can diagnose a run that looks busy and learns nothing. If epsilon is 0 on a fresh table with 4 actions, you never try 3 of them. If the target network is copied every step, the regression label moves as fast as the prediction. If two actions differ by 0.2 and you stopped value iteration while values can still move by 0.5, the greedy action is not settled. Next you plot return against steps on one environment.

Table solvers and deep updates.

Table methods

Value or policy iteration

Q-learning on cells

A gap versus the residual

Deep updates

Replay and a frozen target

Advantage scales the actor

Clip the probability ratio

A flat return you can read

1. A task has 4 actions, discount 0.9, and rewards of 0 or 1. The run uses a deep Q network with epsilon 0 and copies the target every step. After 200 updates the plotted return is still 0.

2. Two mechanisms from earlier lectures are off. Epsilon 0 never tries 3 of the 4 actions once ties break one way. Copying the target every step moves the label with the online weights.

3. The discounted return of rewards at most 1 cannot exceed 1 divided by 0.1, so 10. A later plot that climbs past 10 is a logging error or an undiscounted sum, not a policy that beat the bound.

Problem 1. Epsilon is 0 and the target network is copied every step. Which two mechanisms failed, and what was each supposed to do?

Answer. Exploration and the frozen target. Epsilon-greedy was supposed to try a random action on some steps so every action gets data; at 0 it does not. The frozen target was supposed to hold the regression label still between copies; copying every step lets the label move with the online weights.

Problem 2. Discount is 0.9 and every reward is at most 1. Your plot shows an average discounted return of 12. What does the bound say?

Answer. The return cannot exceed 1 divided by 1 minus 0.9, which is 10. A plotted 12 is a measurement error, an undiscounted sum, or a reward outside the range you assumed.